Функции и Модули в Python

Основные концепции функций, параметров, возврата значений, модулей и библиотек в Python. Вы узнаете, как создавать и вызывать свои собственные функции, работать с модулями стандартной библиотеки, а также создавать простые классы и объекты.

ST СИ Цехош

Дата обращения: 16.11.2025

Объявление и Вызов Функции

1

Объявление Функции

Функция определяется с помощью ключевого слова " def" и имени функции. Внутри функции выполняется логика, которую нужно вызвать.

2

Вызов Функции

Чтобы использовать функцию, нужно вызвать её по имени. После вызова, код внутри функции будет выполнен.

3

Возврат Значения

Функция может возвращать одно или несколько значений с помощью ключевого слова "return ".

def

Функция - это именованный блок кода, к которому можно обратиться из любого места программы. У функции есть имя и список входных параметров, а также возвращаемое значение.

Функция позволяет использовать в программе один и тот же фрагмент кода несколько раз.

def

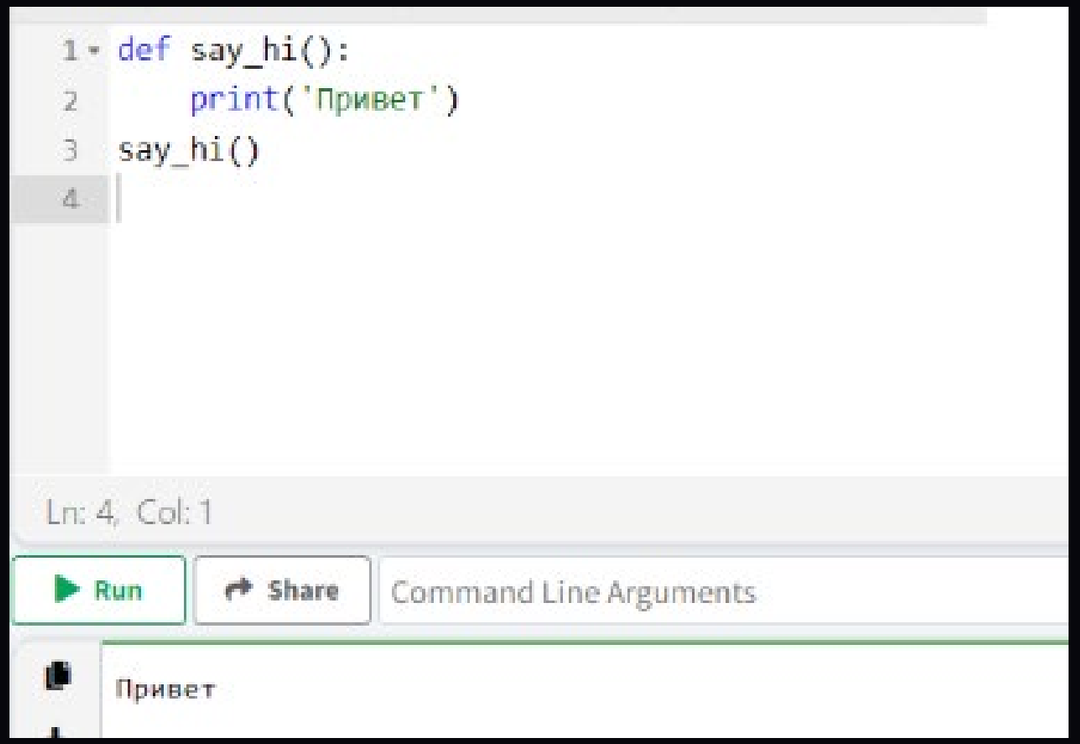
```
def say_hi():
    print ('Привет' )
say_hi()
```

def - DEclare Function - "объявить функцию"

say_hi - название новой функции (функции - рекомендуем называть глаголами).

Код внутри функции должен идти с **табуляцией** . Это весь код, который идет после двоеточия.

название - **say_hi** (отвечает за то, что мы вызываем), **()** - **вызов** функции.



def

```
def function_name(argument1, argument2, ...):
    код функции
```

def - DEclare Function - "объявить функцию"

function_name - имя функции

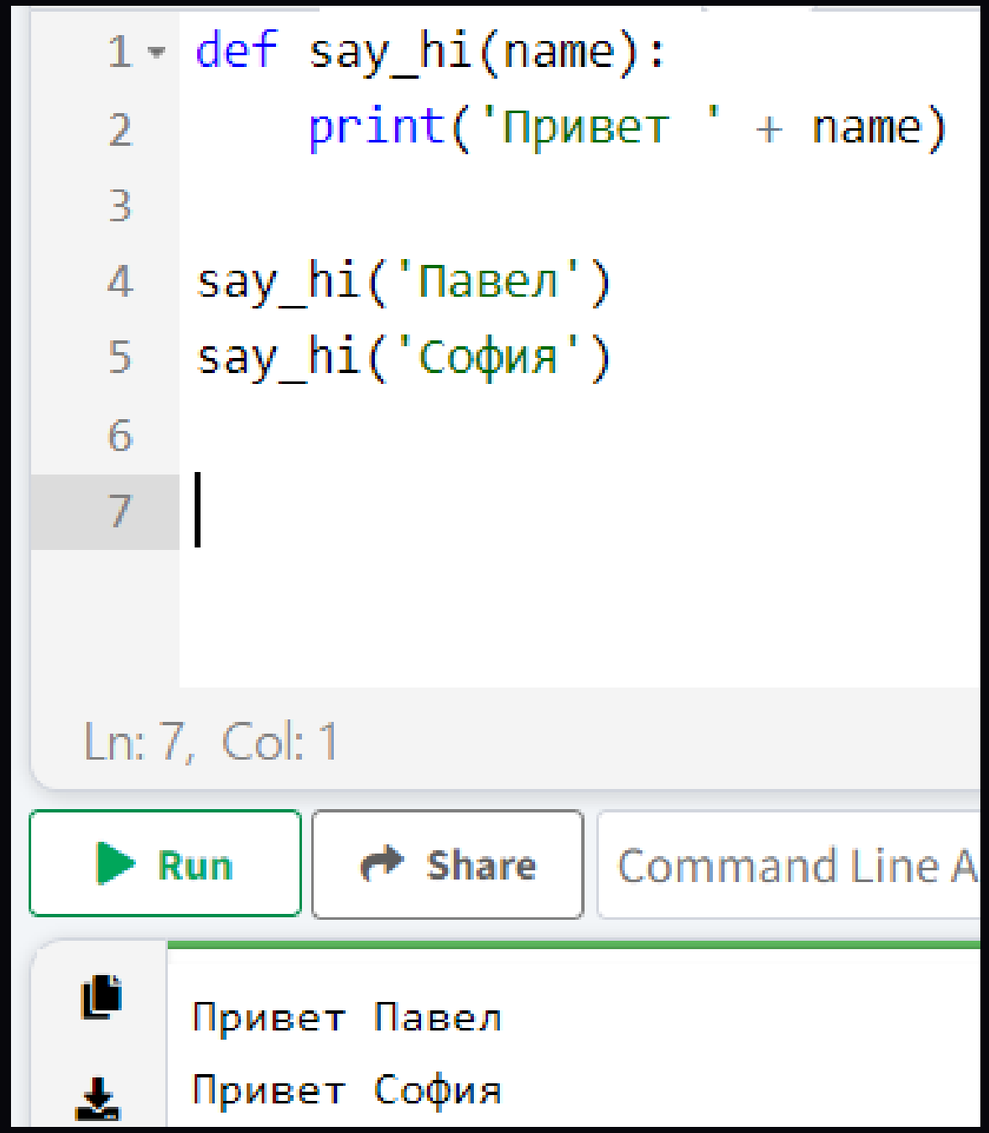
(argument1, argument2, ...) - список аргументов, поступающих на вход функции при ее вызове

Код внутри функции - это весь код, который идет после двоеточия

Функция с одним аргументом.

```
def say_hi(name):
    print('Привет ' + name)

say_hi('Павел')
say_hi('София')
```



def - "объявить функцию"

say_hi - название новой функции

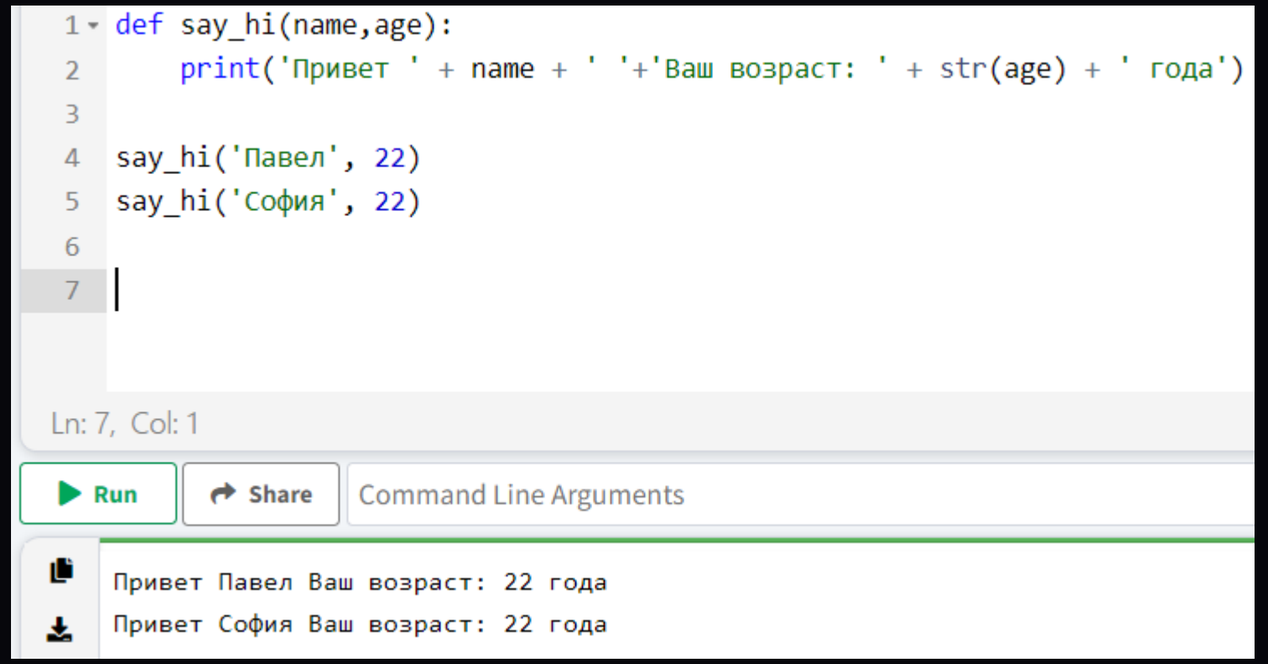
Код внутри функции .

название - **say_hi** (отвечает за то, что мы вызываем), **()** - **вызов** функции.

Функция с **множеством** аргументов.

Можно задавать столько аргументов сколько потребуется

```
def say_hi(name, age):
    print('Привет ' + name + ' '+'Ваш возраст: ' + str(age) + ' года')
say_hi('Павел', 22)
say_hi('София', 22)
```



#Вывод

Привет **Павел** Ваш возраст: **22** года

Привет **София** Ваш возраст: **22** года

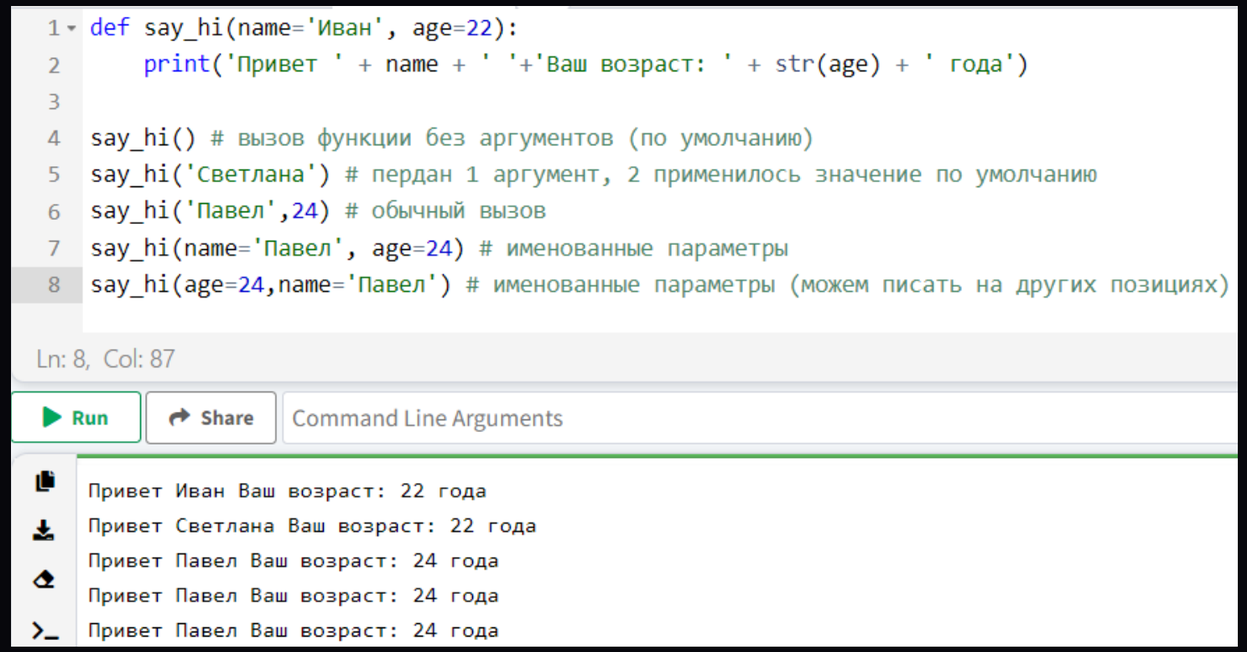
def - "объявить функцию"

say_hi - **название новой функции**

Код внутри функции.

название - **say_hi** (отвечает за то, что мы вызываем), **()** - **вызов** функции.

def



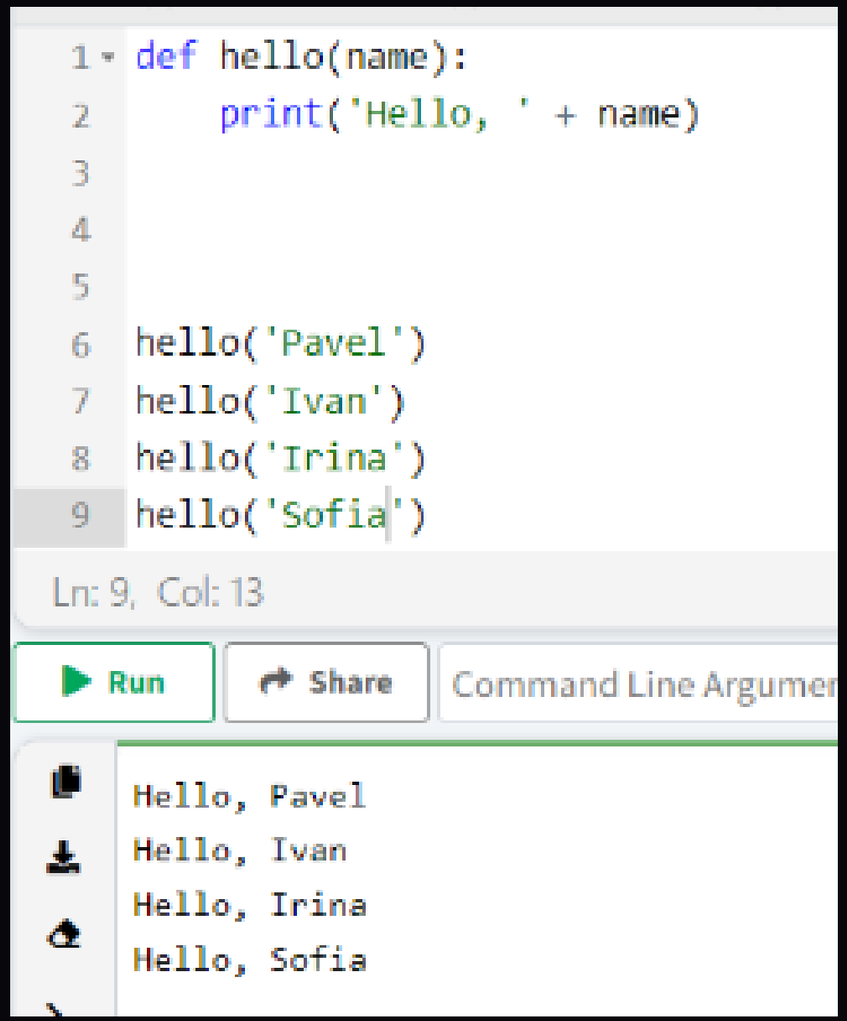

```
def say_hi(name='Иван', age=22):
    print('Привет ' + name + ' '+'Ваш возраст: ' + str(age) + ' года')
say_hi() # вызов функции без аргументов (по умолчанию)
say_hi('Светлана') # передан 1 аргумент, 2 применилось значение по умолчанию
say_hi('Павел',24) # обычный вызов
say_hi(name='Павел', age=24) # именованные параметры
say_hi(age=24,name='Павел') # именованные параметры (можем писать на других позициях)
```

def

```
# Объявление функции

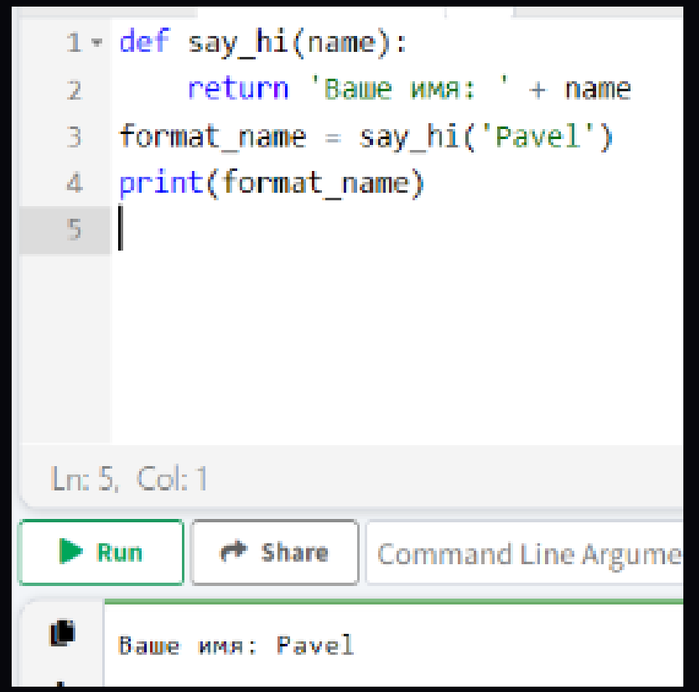

def hello(name):
    print('Hello, ' + name)


# Вызовы функции
hello('Pavel')
hello('Ivan')
hello('Irina')
hello('Sofia')
```



Оператор return

```
def say_hi(name):
    return 'Ваше имя: ' + name
format_name = say_hi('Pavel')
print(format_name)
```



def - "объявить функцию"

say_hi - **название новой функции**

return - позволяет создать функцию, которая возвращает значение

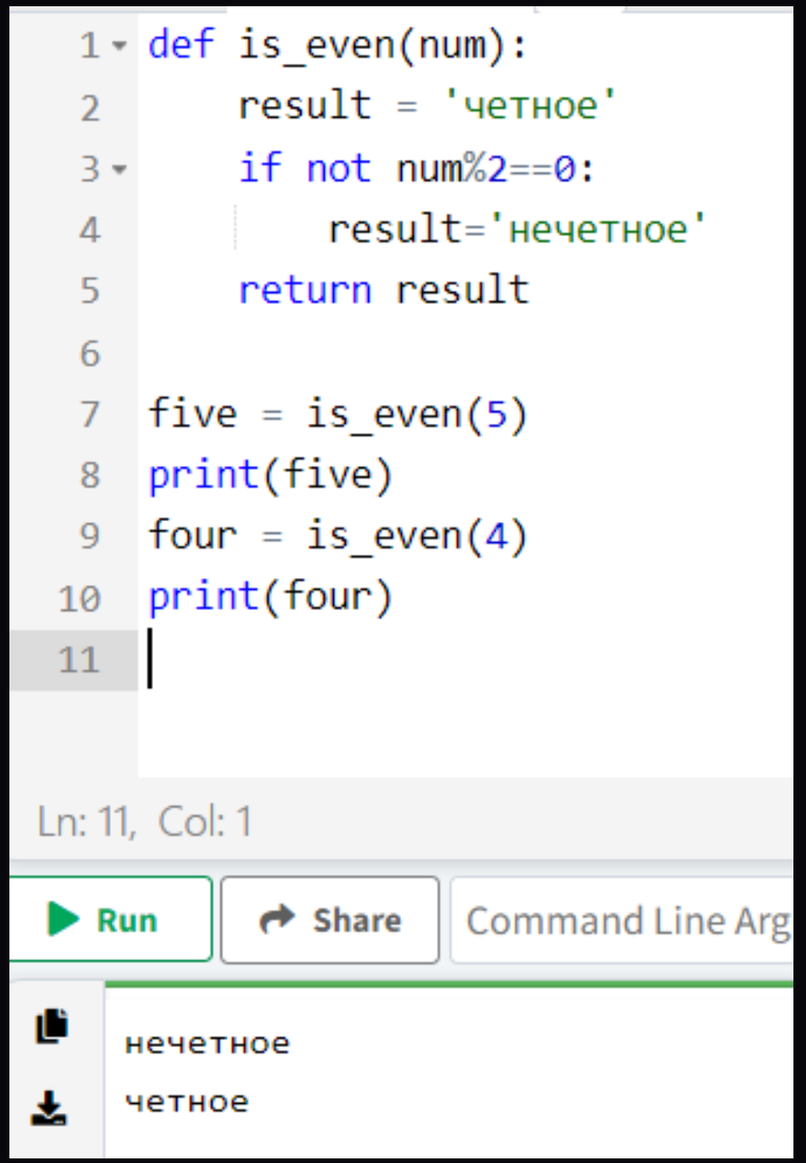
тело функции - это весь код, который идет после двоеточия название

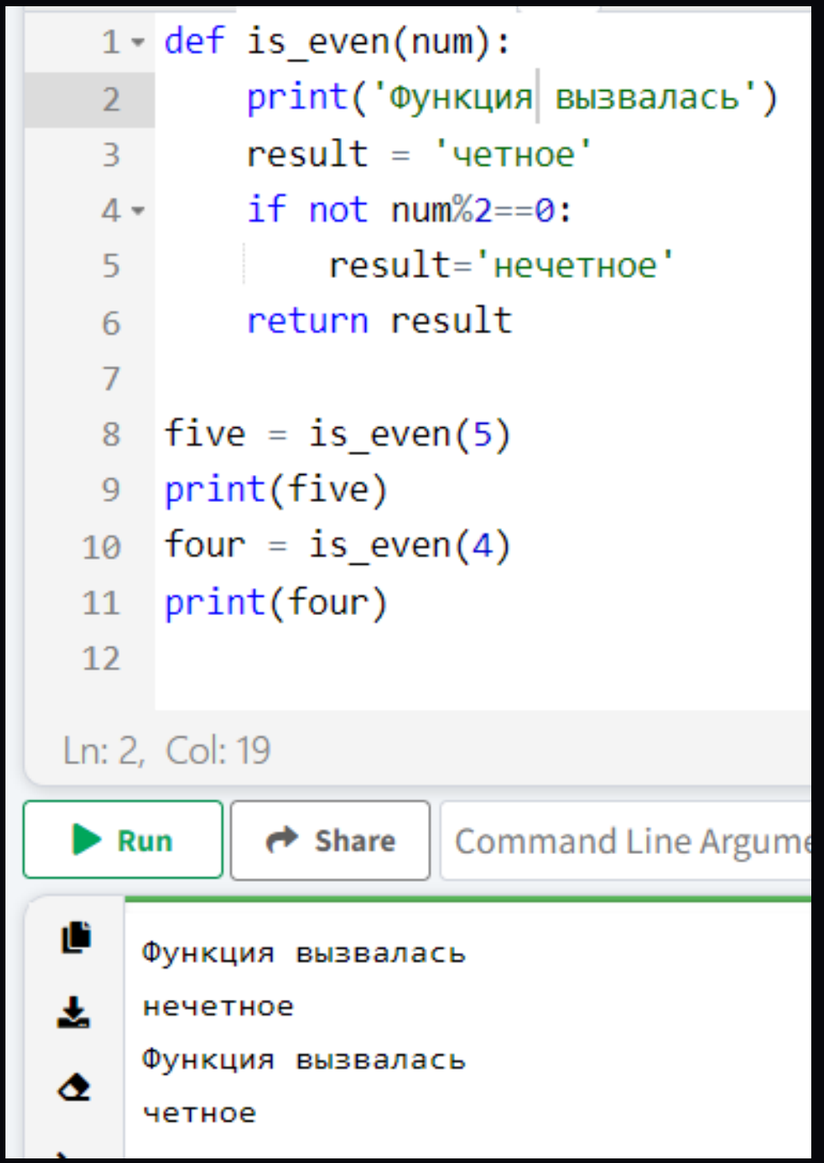
say_hi (отвечает за то, что мы вызываем), **()** - **вызов** функции.

Оператор **return** возвращает значение из функции.

Оператор return

```
def is_even(num ):
    result  = 'четное '
    if not num%2==0:
        result='нечетное'
    return  result
five = is_even(5)
print(five)
four = is_even(4)
print(four)
```





Оператор **return** возвращает значение из функции.

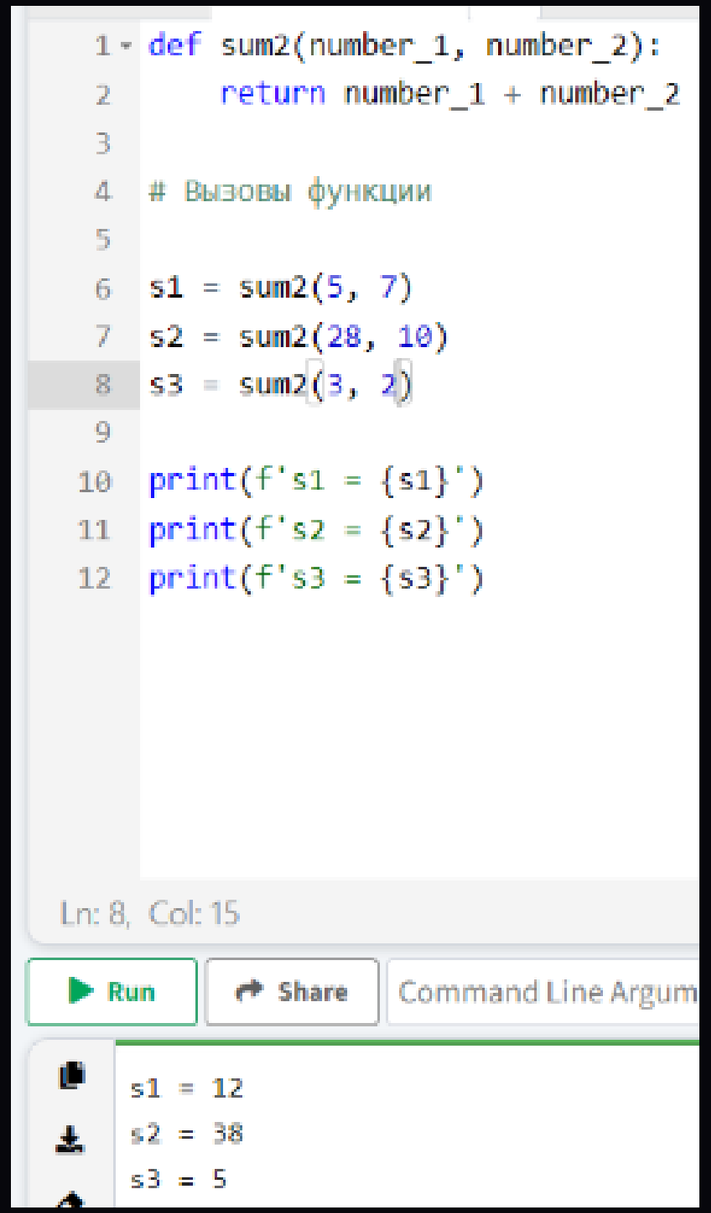
Оператор return

```
# Объявление функции
def sum2(number_ 1, number_ 2):
    return  number_ 1 + number_ 2


# Вызовы функции
s1 = sum2(5, 7)
s2 = sum2(28, 10)
s3 = sum2(3, 2)


print(f's1 = {s1}')
print(f's2 = {s2}')
print(f's3 = {s3}')


#Вывод
s1 = 12
s2 = 38
s3 = 5
```



Оператор **return** возвращает значение из функции.

```
questions = ['Какой фон имеют временные дорожные знаки?\n1.Желтный фон.\n2.Синий фон.\n3.Оранжевый фон','На каком расстояние...']
correct_answers = [3, 2]
explanation = ['Установлено ГОСТом\n', 'Принято ГОСТом\n ']
user_answers = []
def is_correct(user_answers,correct_answers):
    count =0
    for answer in user_answers:
        if answer == correct_answers[user_answers.index(answer)]:
            print('Правильный ответ')
            count += 1
        else:
            print('Неправильный ответ')
            print(explanation[user_answers.index(answer)])
    print(f"Результат: {count} / {len(user_answers)} ")
def print_question(question):
    print(question)
    user_input= int(input('Введите Ваш ответ: '))
   user_answers.append(user_input)
for question in questions:
    print_question(question)
print('Подсчёт реультатов: ')
is_correct(user_answers,correct_answers)
```

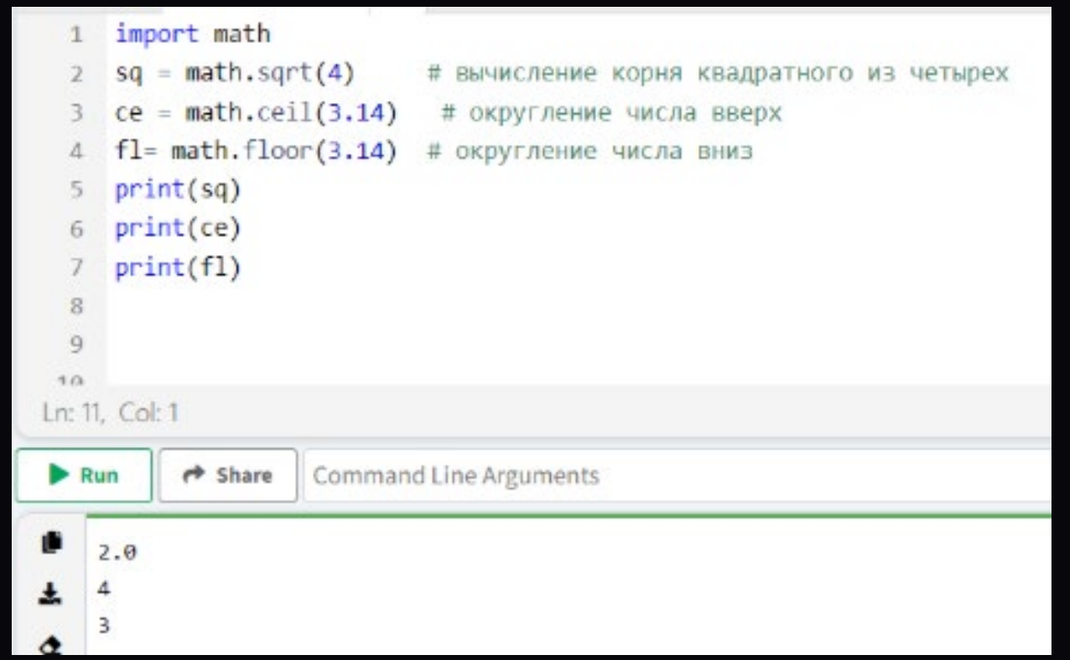
Пример теста

math

Модуль **math** – набор функций для выполнения различных математических операций.

Для использования функций модуля math в начале программы его необходимо импортировать командой **import**.

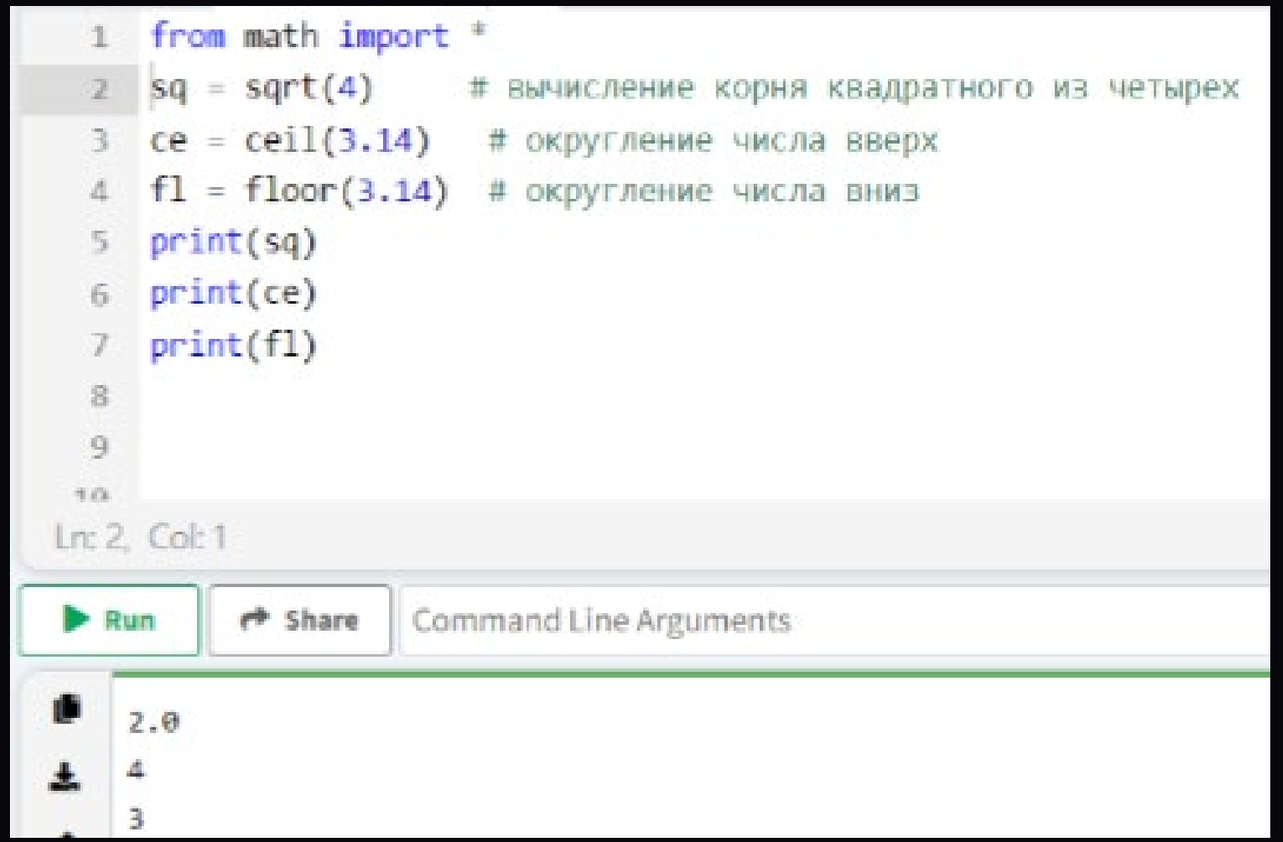
```
import math
sq = math.sqrt(4)     # вычисление корня квадратного из четырех
ce = math.ceil(3.14)   # округление числа вверх
fl= math.floor(3.14)  # округление числа вниз
print(sq)
print(ce)
print(fl)
```



Обращаемся к библиотеки, чтобы вызвать метод math.sqrt(4)

math

```
from math import *
sq = sqrt(4)     # вычисление корня квадратного из четырех
ce = ceil(3.14)   # округление числа вверх
fl = floor(3.14)  # округление числа вниз
print(sq)
print(ce)
print(fl)
```



* - прописываем только название функции. Не обращаясь к имени библиотеки

math

```
1 from math import sqrt, ceil, floor
2 print(sqrt(4))
3 print(ceil(3.14))
4 print(floor(3.14))
```

Ln: 4, Col: 17

Run | Share | Command Line Arguments

```
2.0
4
3
```

from math import sqrt, ceil, floor # из math импортировать конкретные функции

print(sqrt(4))

print(ceil(3.14))

print(floor(3.14))

```
1 from math import sqrt, floor
2 print(sqrt(4))
3 print(ceil(3.14))
4 print(floor(3.14))
```

Ln: 3, Col: 19

Run | Share | Command Line Arguments

```
2.0
Traceback (most recent call last):
  File "Untitled21.py", line 3, in <module>
    print(ceil(3.14))
NameError: name 'ceil' is not defined
```

from math import sqrt, floor # из math импортировать конкретные функции

print(sqrt(4))

print(ceil(3.14)) # приведет к ошибке, так как **функция ceil не подключена**

print(floor(3.14))

datetime

В Python работа с датами и временем осуществляется с помощью модуля **datetime**. Этот модуль предоставляет классы для работы с датами, временем, а также для выполнения различных операций с ними.

datetime

Основные классы модуля datetime:

- ✓ **datetime.date** - представляет дату (год, месяц, день).
- ✓ **datetime.time** - представляет время (часы, минуты, секунды, микросекунды).
- ✓ **datetime.datetime** - объединяет дату и время.
- ✓ **datetime.timedelta** - представляет разницу между двумя датами или временами.
- ✓ **datetime.tzinfo** - базовый класс для реализации временных зон.

Импорт модуля

Для работы с модулем нужно его импортировать:

```
import datetime
```

datetime

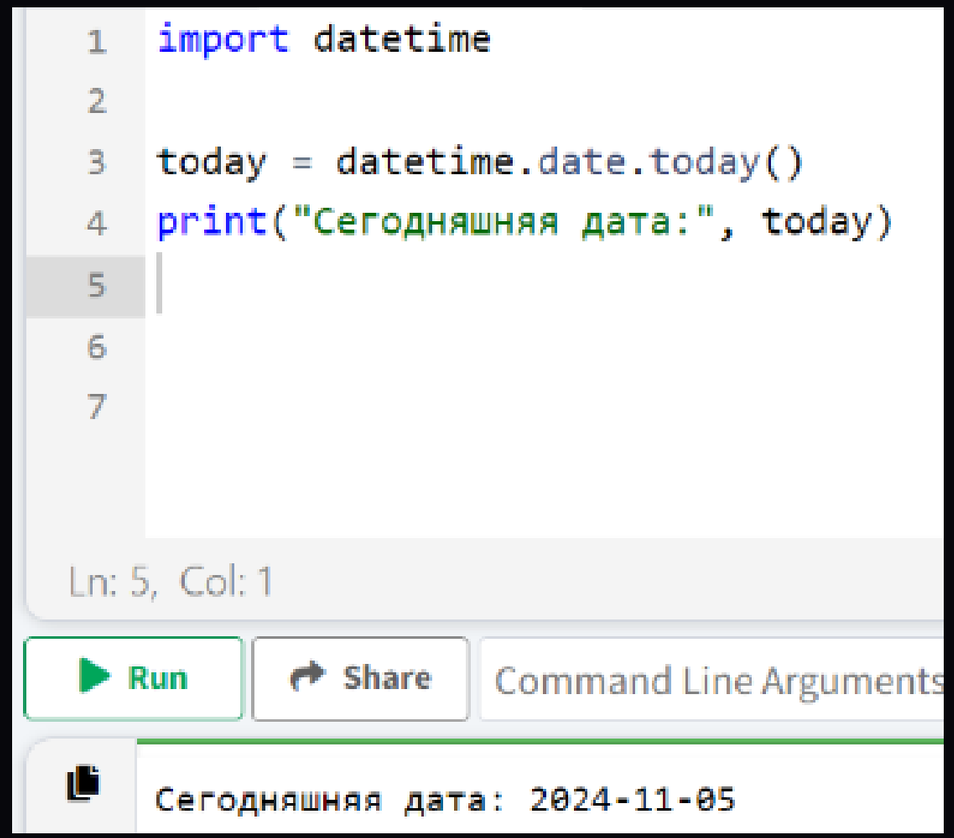
Пример

Создание объекта даты:

```
import datetime


today = datetime.date.today()
print("Сегодняшняя дата:", today)


#Вывод
Сегодняшняя дата: 2024-11-05
```



datetime.date - представляет дату (год, месяц, день).

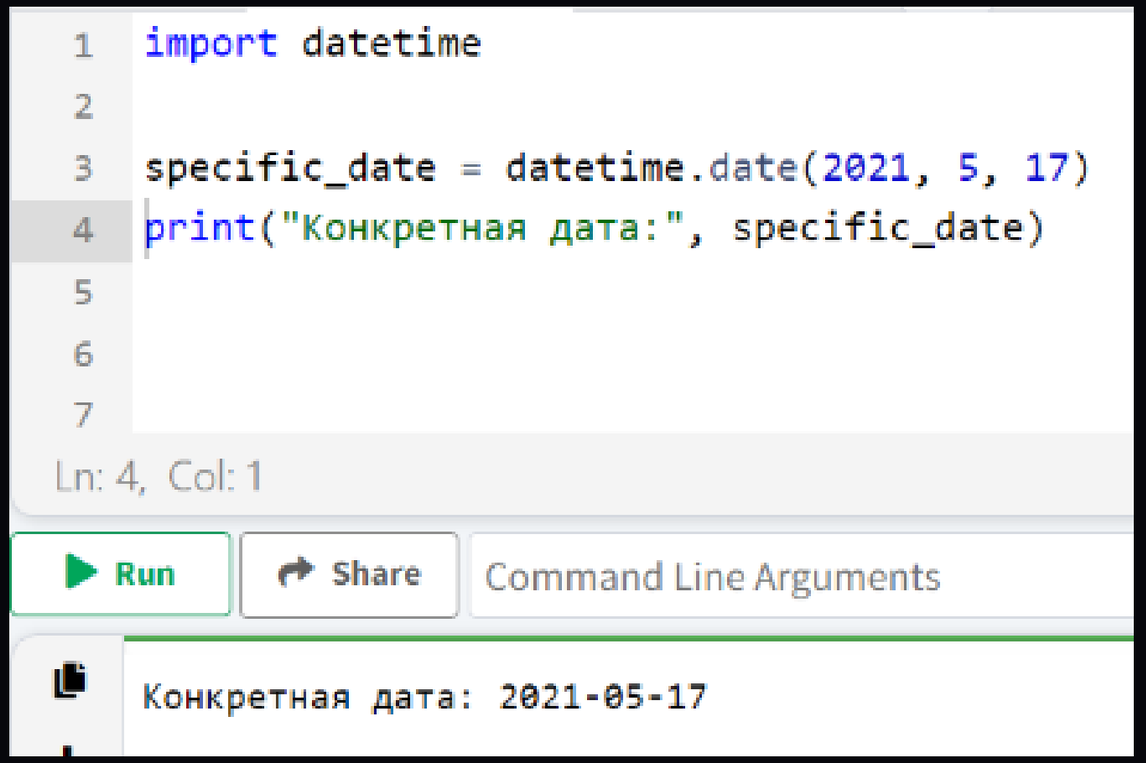
Пример

```
import datetime

specific_date = datetime.date(2021, 5, 17)
print("Конкретная дата:", specific_date)

#Вывод

Конкретная дата: 2021-05-17
```



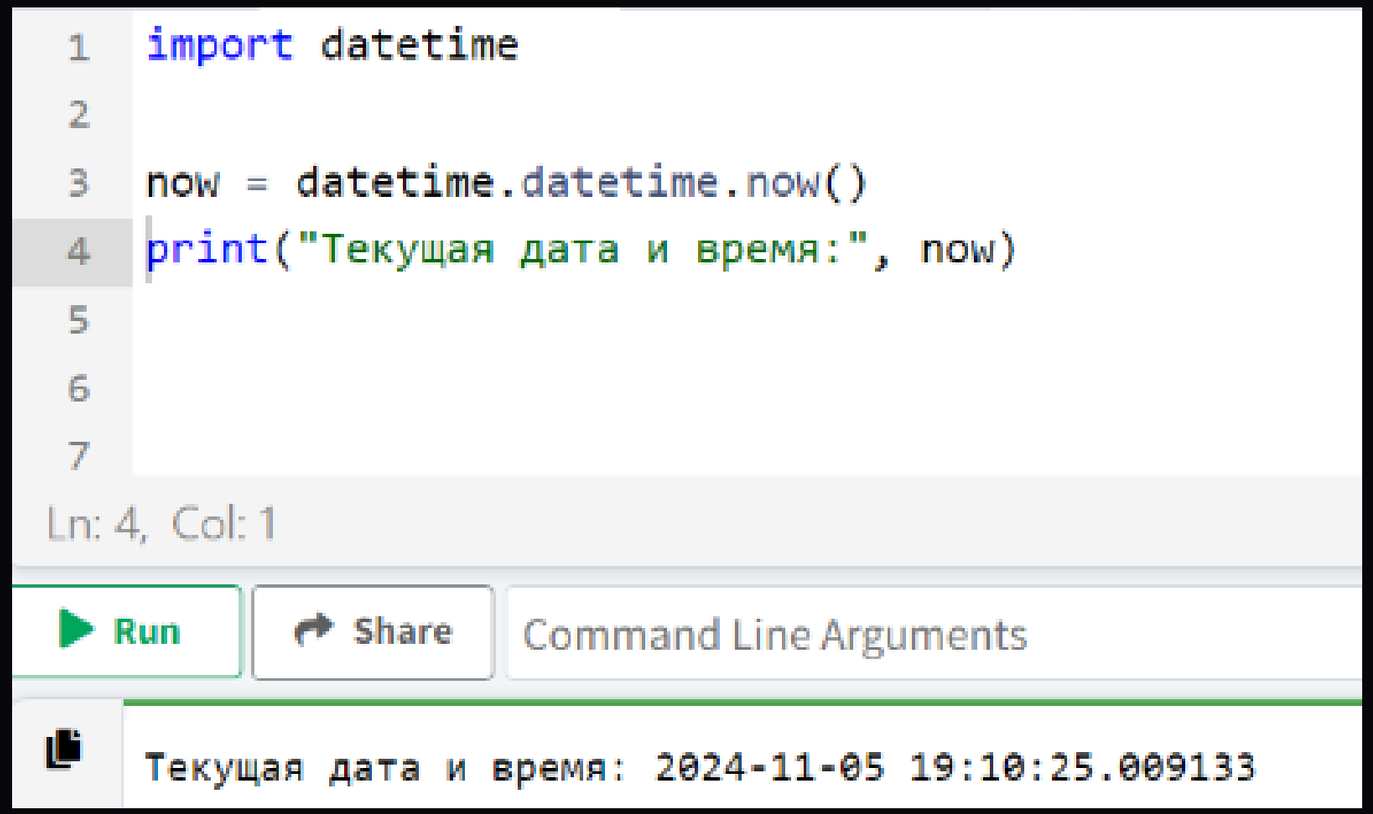
datetime.date - представляет дату (год, месяц, день).

Пример

```
import datetime

now = datetime.datetime.now()
print("Текущая дата и время:", now)

#Вывод
Текущая дата и время: 2024-11-05 19:10:25.009133
```



datetime.datetime - объединяет дату и время.

Форматирование даты и времени

Для форматирования даты и времени используется метод strftime:

```
import datetime

now = datetime.datetime.now()

formatted_date = now.strftime("%Y-%m-%d %H:%M:%S")

print("Отформатированная дата:", formatted_date)


#Вывод

Отформатированная дата: 2024-11-05 19:22:07
```

strftime

Основные коды форматирования:

%Y — год с четырьмя цифрами (например, 2024).

%y — год с двумя цифрами (например, 24).

%m — месяц как целое число (01-12).

%B — полное название месяца (например, Январь).

%b — сокращенное название месяца (например, Ян).

%d — день месяца (01-31).

%H — час в 24-часовом формате (00-23).

%I — час в 12-часовом формате (01-12).

%M — минуты (00-59).

%S — секунды (00-59).

%p — AM или PM.

%A — полное название дня недели (например, Понедельник).

%a — сокращенное название дня недели (например, Пн).

strftime

Пример

```
import datetime

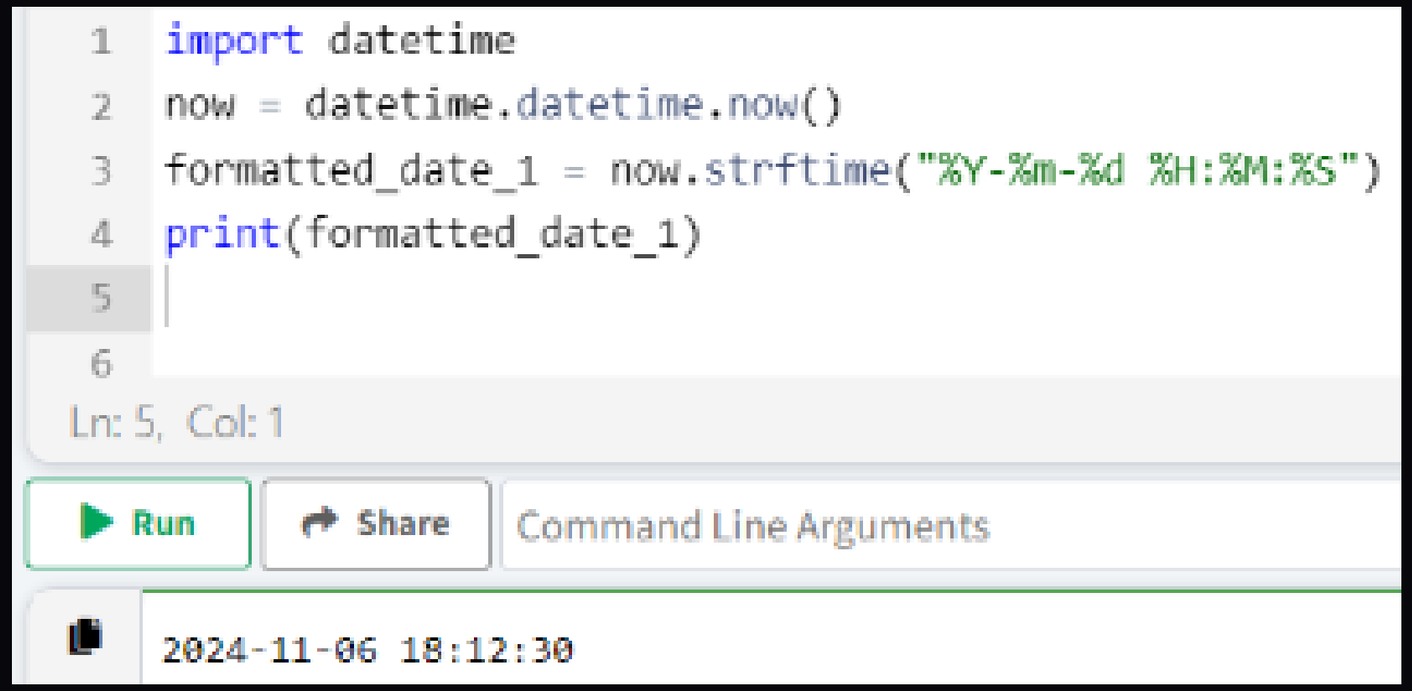
now = datetime.datetime.now()

formatted_date_1 = now.strftime("%Y-%m-%d %H:%M:%S")

print(formatted_date_1)


# Вывод

2024-11-06 18:12:30
```



Пример

```
import datetime

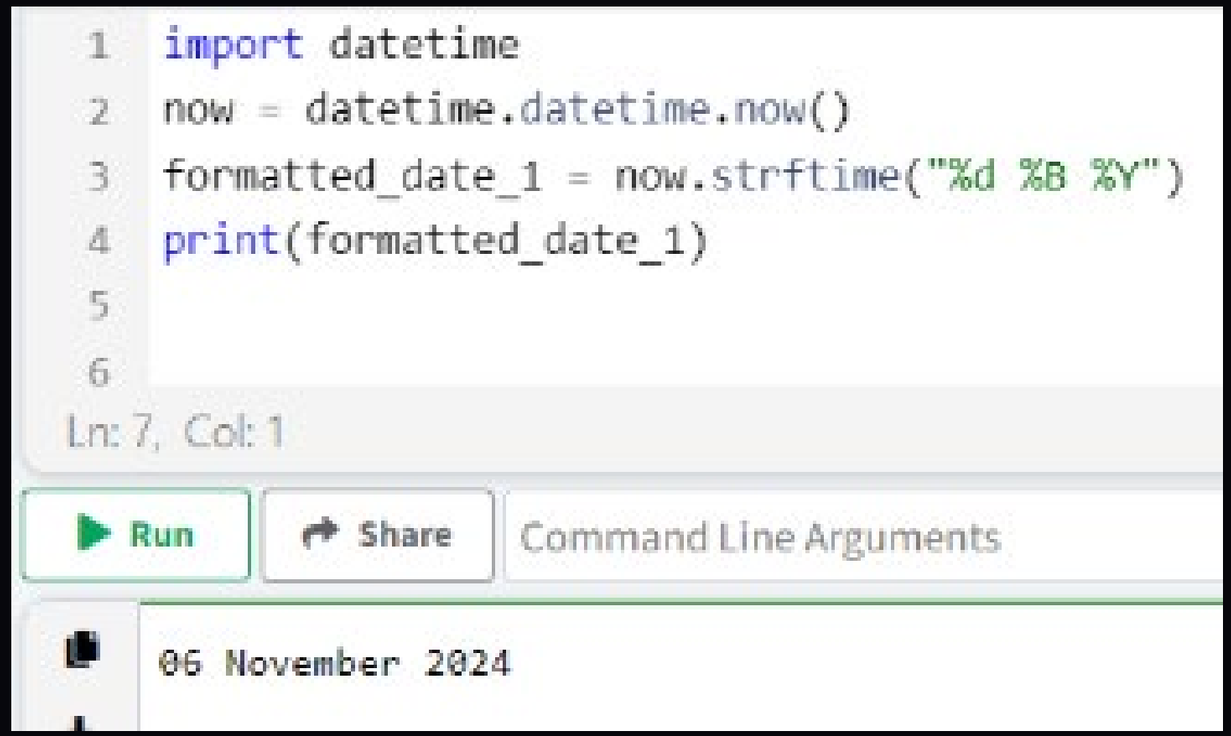
now = datetime.datetime.now()

formatted_date_1 = now.strftime("%d %B %Y")

print(formatted_date_1)


# Вывод

06 November 2024
```



Пример

```
import datetime

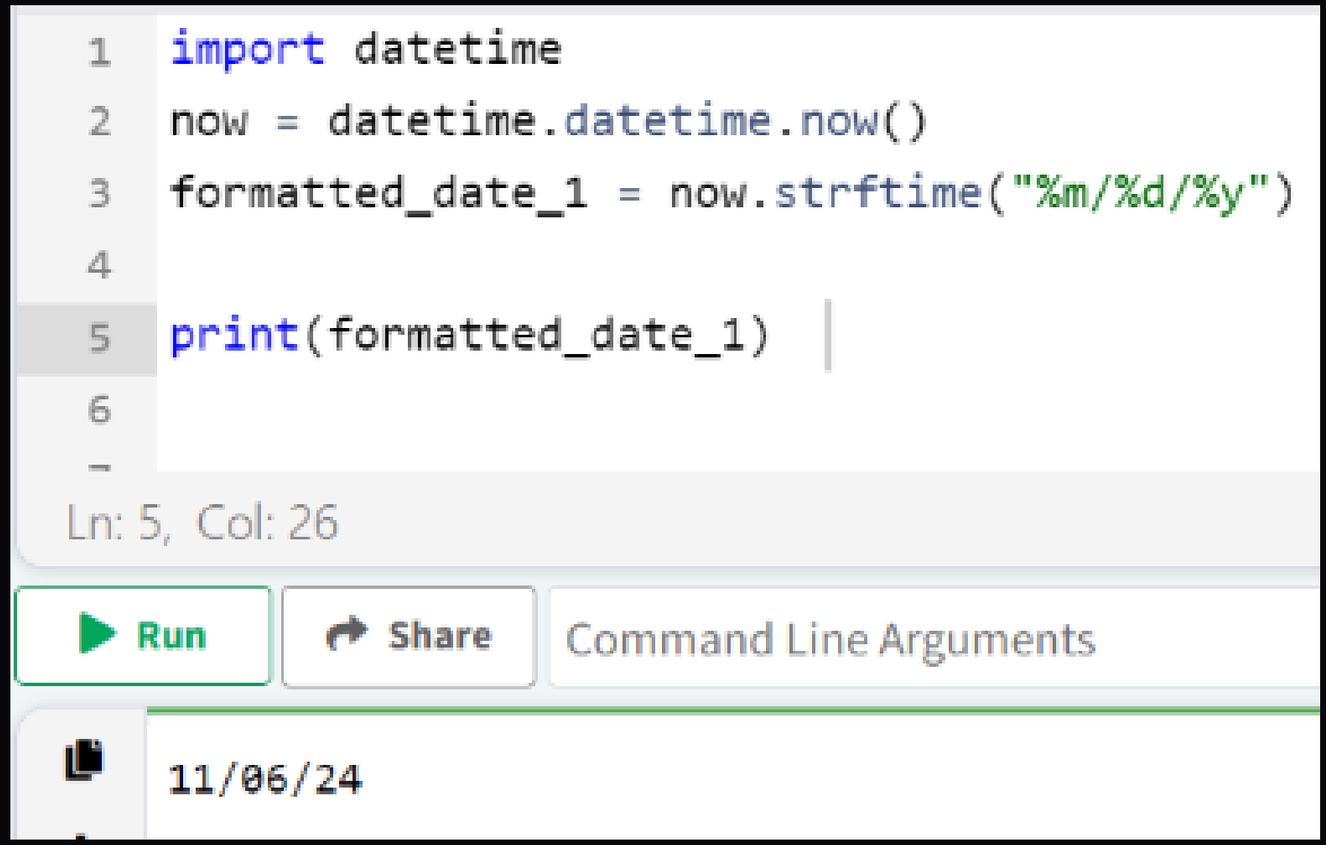
now = datetime.datetime.now()

formatted_date_1 = now.strftime("%m/%d/%y")

print(formatted_date_1)


# Вывод

11/06/24
```



Пример

```
import datetime

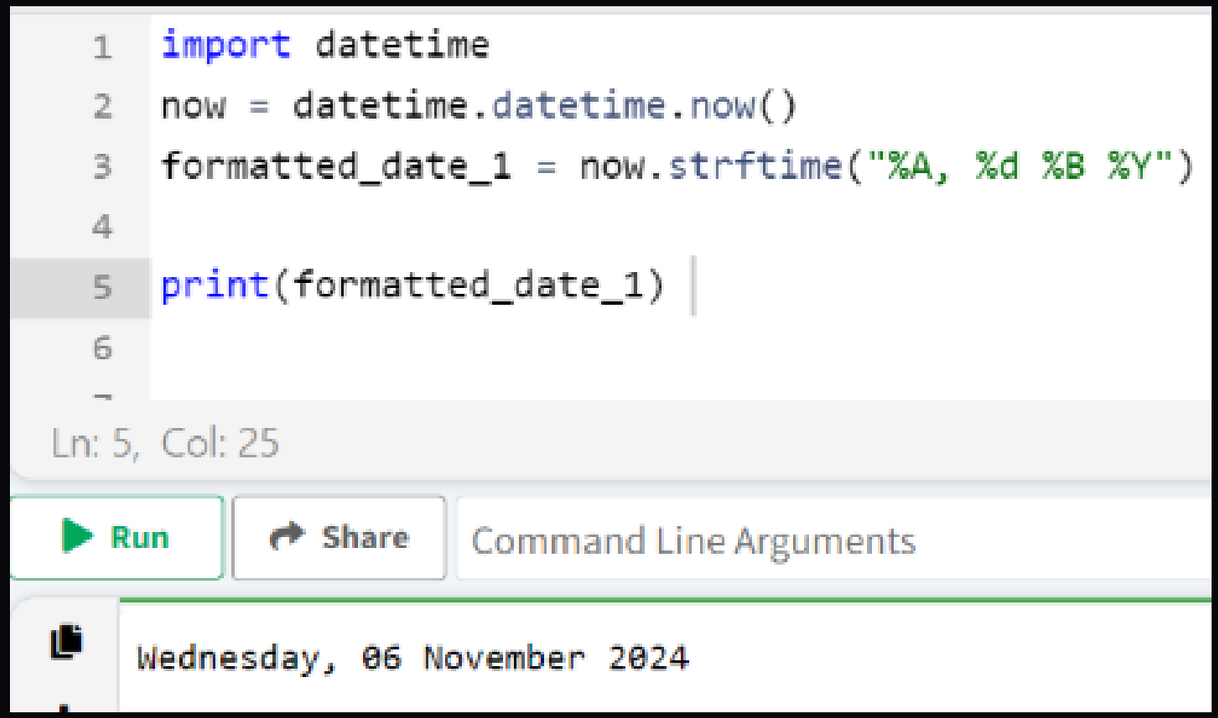
now = datetime.datetime.now()

formatted_date_1 = now.strftime("%A, %d %B %Y")


print(formatted_date_1)

# Вывод

Wednesday, 06 November 2024
```



Пример

```
import datetime

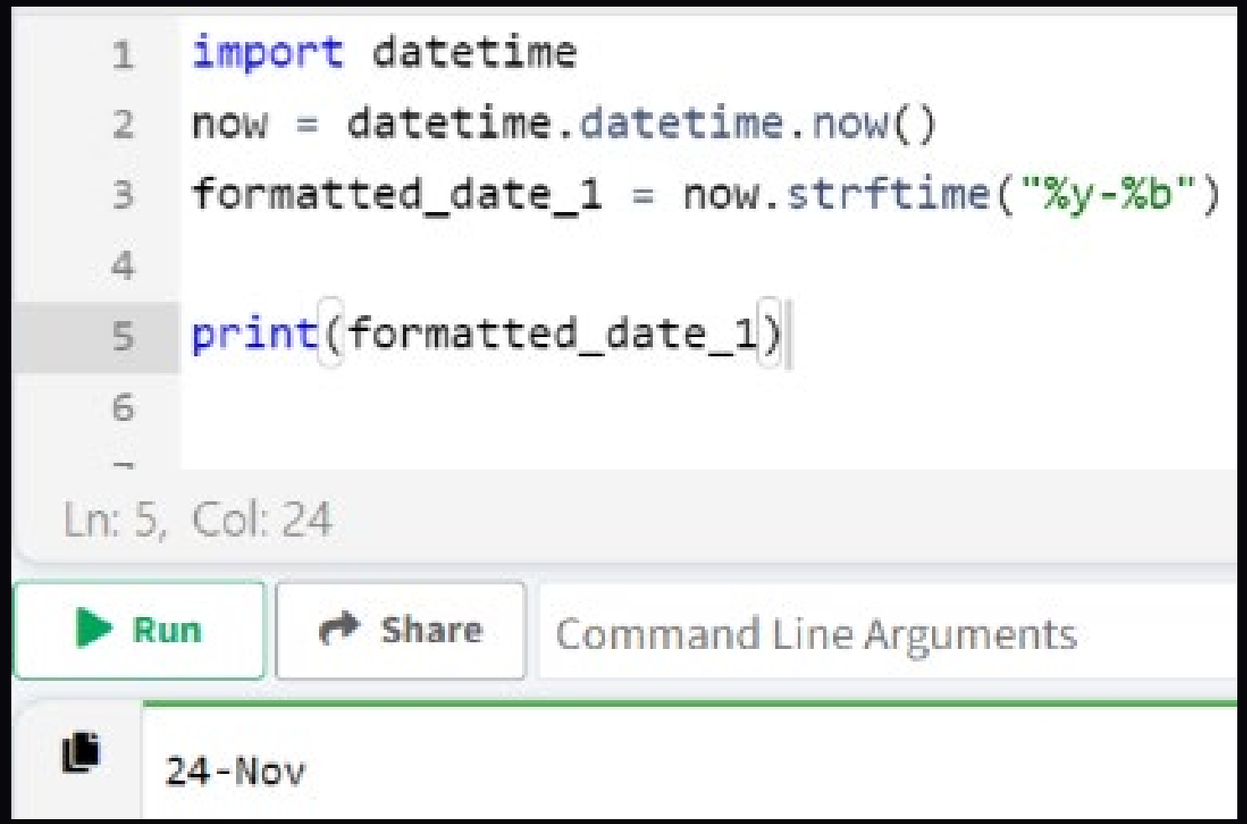
now = datetime.datetime.now()

formatted_date_1 = now.strftime("%y-%b")


print(formatted_date_1)

# Вывод

24-Nov
```



Пример

```
import datetime

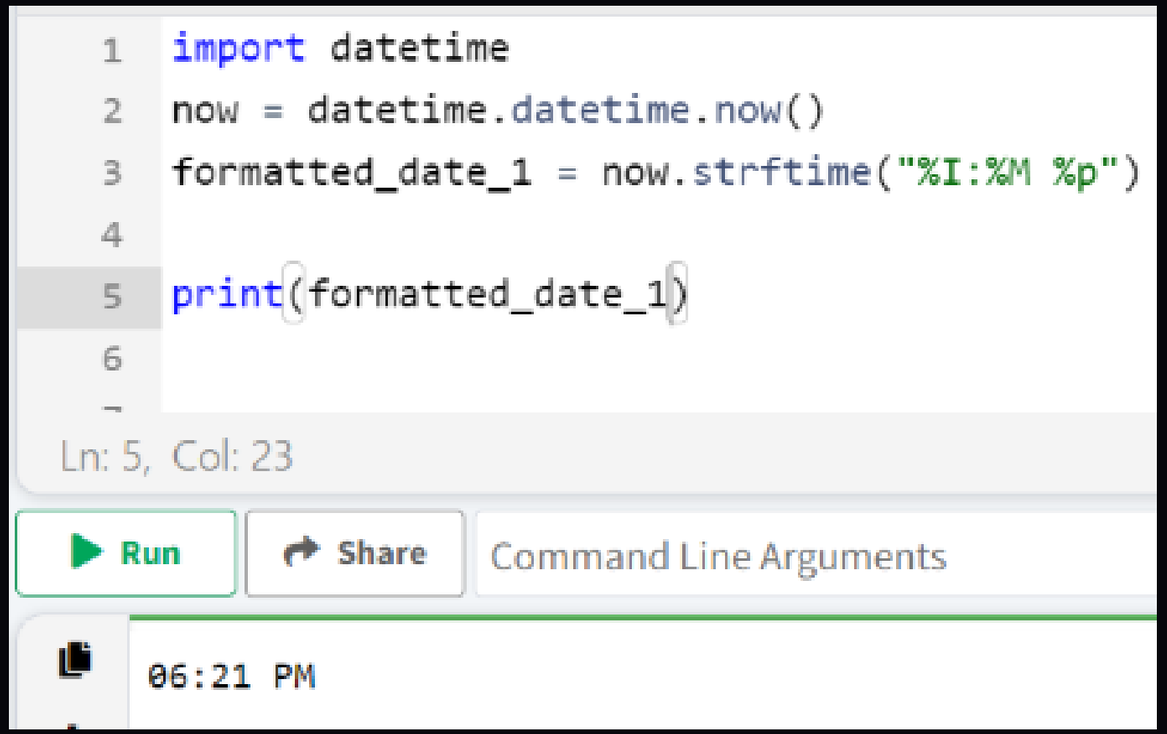
now = datetime.datetime.now()

formatted_date_1 = now.strftime("%I:%M %p")

print(formatted_date_1)


# Вывод

06:21 PM
```



Пример

```
import datetime

date_string = "2023-10-05 14:25:36"

parsed_date = datetime.datetime.strptime(date_string, "%Y-%m-%d %H:%M:%S")

print("Парсинг даты из строки:", parsed_date)


# Вывод
```

Пример

```
import datetime

date1 = datetime.date(2024, 10, 5)

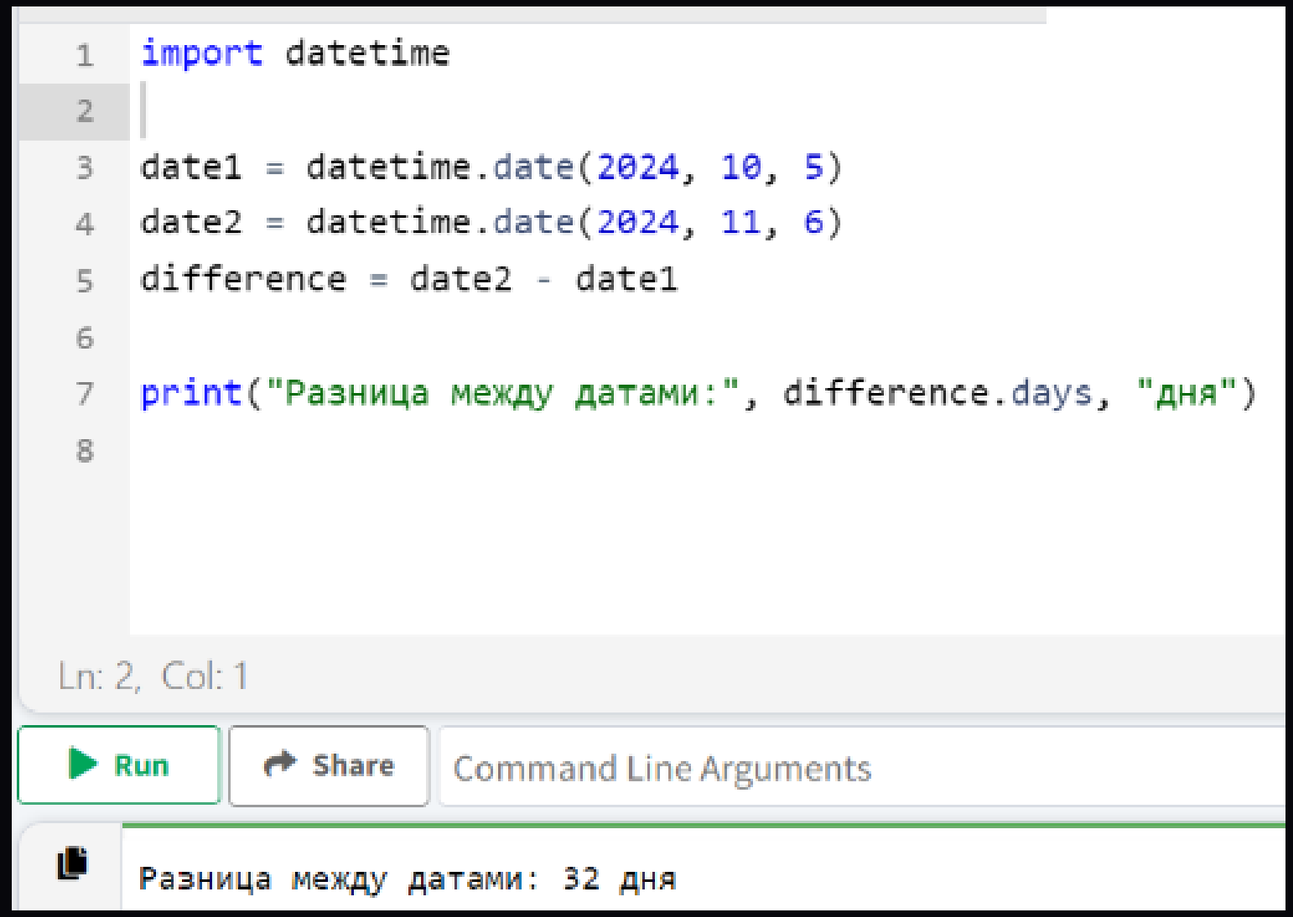
date2 = datetime.date(2024, 11, 6)

difference = date2 - date1

print("Разница между датами:", difference.days, "дня")

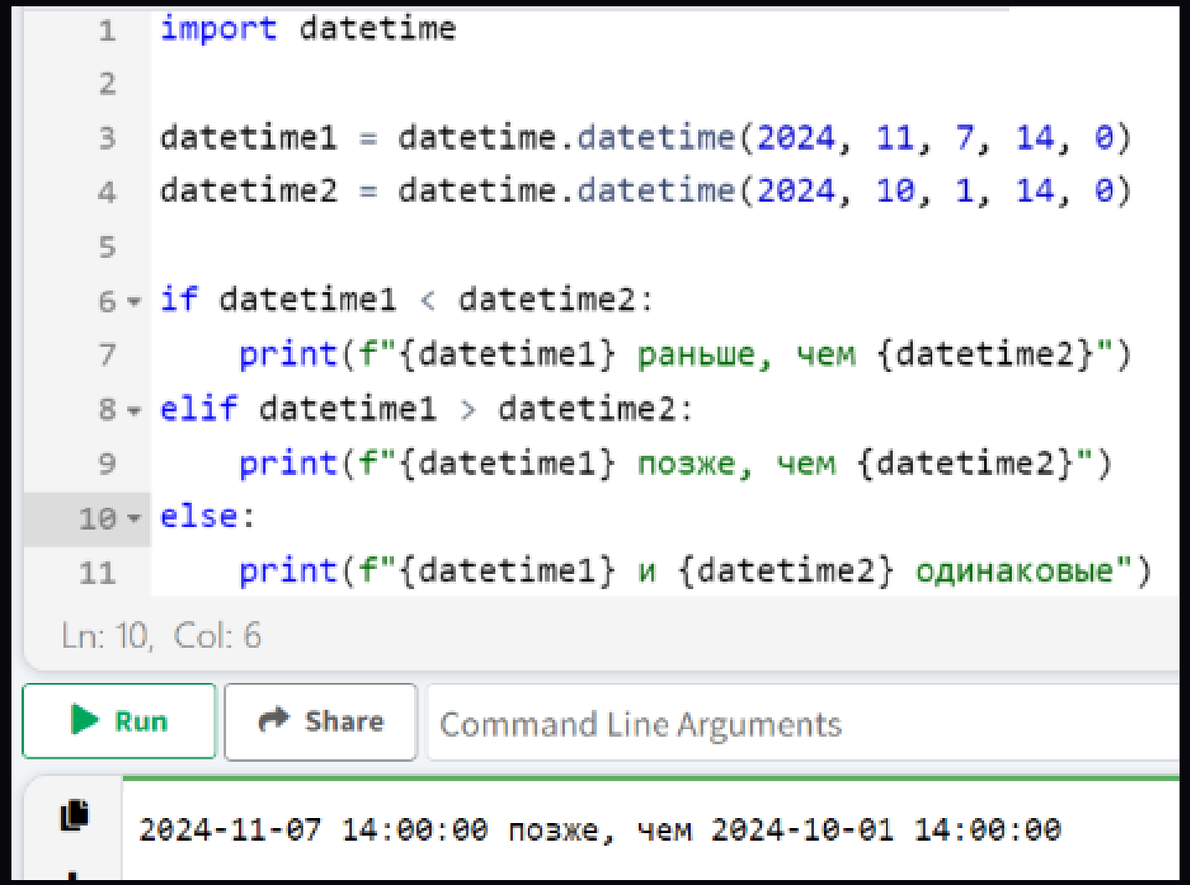

# Вывод

Разница между датами: 32 дня
```



```
import datetime


datetime1 = datetime.datetime(2024, 11, 7, 14, 0)
datetime2 = datetime.datetime(2024, 10, 1, 14, 0)
if datetime1 < datetime2: # Сравнение дат и времени
    print(f"{datetime1} раньше, чем {datetime2}")
elif datetime1 > datetime2:
    print(f"{datetime1} позже, чем {datetime2}")
else:
    print(f"{datetime1} и {datetime2} одинаковые")
```



datetime.datetime